The FABRIC Strategy for Verifying Neural Feedback Systems

I. Samuel Akinwande[1] Sydney Katz[1] Mykel Kochenderfer[1] Clark Barrett[2]

***Abstract*— Forward reachability analysis is a dominant approach for verifying reach–avoid specifications in neural feedback systems, i.e., dynamical systems controlled by neural networks, and a number of directions have been proposed and studied. In contrast, far less attention has been given to backward reachability analysis for these systems, in part because of the limited scalability of known techniques. In this work, we begin to address this gap by introducing new algorithms for computing both over- and underapproximations of backward reachable sets for nonlinear neural feedback systems. We also describe and implement an integration of these backward reachability techniques with existing ones for forward analysis. We call the resulting algorithm Forward and Backward Reachability Integration for Certification (FaBRIC). We evaluate our algorithms on a representative set of benchmarks and show that they significantly outperform the prior state of the art.**

I. Introduction

Dynamical systems controlled by neural networks are becoming increasingly prevalent, with applications including robotics [1] and autonomous systems [2]. Following [3], we call them *neural feedback systems* (NFS). As more of these systems are deployed, it is essential to be able to verify that they satisfy intended specifications under all relevant operating conditions.

Existing approaches for verifying neural feedback systems include sampling-based methods [4], certificate synthesis [5], and reachability analysis [3]. Reachability analysis aims to characterize the set of all system states that can be reached to determine whether a given specification is satisfied or violated [6]. In *forward* reachability, one computes the reachable set of states starting from a given initial set over a specified time horizon, whereas *backward* reachability characterizes the set of states from which the system can be driven into a target set. The two fail in complementary ways. Forward analysis is cheap, but its error compounds, leading to costly later steps. Backward analysis is simply too expensive for practical use. Most reachability analysis methods for neural feedback systems focus on forward analysis, in part because computing backward reachable sets requires propagating sets backward through a neural network, a task that is known to be challenging [7]. The difficulty is exacerbated when the NFS has nonlinear transition dynamics, as the analysis must then account for nonlinearity in both the neural network and the system dynamics. The majority of work on backward reachability for neural feedback systems focuses on systems with linear transition dynamics [7], [8]. Hybrid zonotope methods, for example, compute exact backward reachable sets and avoid the conservatism of interval-based enclosures, but assume a linear plant, and their extension to nonlinear plants remains open [9]. Approaches that consider systems with nonlinear transition dynamics tend to scale poorly [7]. Everett et al. [10] minimize some of the limitations of backward reachability analysis of neural feedback systems by introducing a domain refinement scheme.

In this work, we explore a verification strategy that combines forward analysis with new techniques for backward analysis to mitigate the limitations of both approaches. We trade the tightness of more expressive set representations for axis-aligned enclosures that admit a MILP encoding and scale to nonlinear dynamics. We first define an algorithm for computing polyhedral overapproximations of backward reachable sets for nonlinear neural feedback systems. The enclosure abstraction is taken from [3] and the domain refinement loop from [10]. Our contribution extends that loop to nonlinear dynamics and a symbolic k-step formulation. We then introduce three algorithms for computing underapproximations of backward reachable sets that explore different trade-offs between scalability and precision: one prioritizes scalability, one prioritizes precision, and one balances both. We evaluate these backward reachability algorithms on a representative benchmark suite and demonstrate significant improvements over the prior state of the art for both over- and underapproximation. Finally, we introduce the FABRIC algorithm, which combines forward and backward reachability analysis by adapting existing bidirectional analysis ideas to the setting of neural feedback systems, and we show that it improves the performance of reachability analysis on neural feedback benchmarks.

II. Background

We denote the set of natural numbers as $\mathbb{N}$, the set of real numbers as $\mathbb{R}$, and the set of non-negative real numbers as $\mathbb{R}_+$. We use $[i..j]$ to represent the set $\{z \in \mathbb{Z} \mid i \leq z \leq j\}$ and $[n]$ to abbreviate $[1..n]$. We write $[x, y]$ for the set $\{r \in \mathbb{R} \mid x \leq r \leq y\}$. If $\vec{S}$ is a finite *sequence*, we write $\vec{S}[i]$ for its ith element. If a sequence is used where a set is expected, the meaning is the set of elements in the sequence (e.g., $s \in \vec{S}$ means that s occurs in the sequence $\vec{S}$). We use bold face for both vectors and sequences and treat them as interchangeable.

We use $\preccurlyeq$ and $\succcurlyeq$ to describe element-wise inequalities for finite sequences, defined as follows. If $\vec{x}$ and $\vec{y}$ are sequences of size n, then so is $\vec{x} \preccurlyeq \vec{y}$, with $(\vec{x} \preccurlyeq \vec{y})[i] = 1$, if $\vec{x}[i] \leq \vec{y}[i]$, and 0 otherwise. We define $\vec{x} \succcurlyeq \vec{y}$ similarly.We write $\mathbf{0}$ for the zero vector, and $\mathbf{1}$ for the vector of all ones. The sizes

[1]Department of Aeronautics and Astronautics, Stanford University, Stanford, CA 94305, USA samakin@stanford.edu

[2]Department of Computer Science, Stanford University, Stanford, CA 94305, USA

of these vectors will be left implicit when it is clear from context. If $\vec{F}$ is a sequence of functions, then $\vec{F}(x)$ is the sequence $(F[1](x), F[2](x), \dots)$.

A. Neural Feedback Systems

We define a discrete-time neural feedback system $\mathcal{N}$ as the tuple

$$\langle n, D, I, \vec{F}, E, \vec{u}, \delta, T, G, A \rangle,$$

where $n \in \mathbb{N}$ is the *dimension* of the system (i.e., each state is an element of $\mathbb{R}^n$), $D \subseteq \mathbb{R}^n$ is the *operating domain*—the region over which the system is to certified. We assume that it contains the the subset of $\mathbb{R}^n$ in which we expect the system to remain during operation and that the controller is defined over this region. $I \subseteq D$ is the set of *initial states*, $\vec{F} = (f_1, \dots, f_n)$ is a sequence of *state transition functions* with $f_i : \mathbb{R}^n \to \mathbb{R}$, $E \subseteq \mathbb{R}^n$ is the *perturbation set* (i.e., the set of admissible disturbance terms that may affect the next state), $\vec{u} : \mathbb{R}^n \to \mathbb{R}^n$ is the *control function*, $\delta \in \mathbb{R}_+$ is the *time-step size*, $T \in \mathbb{N}$ is the *number of time steps*, $G \subseteq \mathbb{R}^n$ is the set of *goal states*, and $A \subseteq \mathbb{R}^n$ is the set of *avoid states* (i.e., unsafe states).

We assume that states evolve over a sequence of T discrete time steps, each of duration δ, giving a total time horizon of $\delta \cdot T$. For a current state $\vec{x} \in \mathbb{R}^n$, the next-state function $next^{\mathcal{N}} : \mathbb{R}^n \to 2^{\mathbb{R}^n}$ defines the possible successor states of $\mathcal{N}$. This set-valued mapping captures the nondeterminism introduced by the perturbation term:

$$next^{\mathcal{N}}(\vec{x}) = \left\{ \vec{x} + \left(\vec{F}(\vec{x}) + \vec{u}(\vec{x}) + \vec{\epsilon} \right) \cdot \delta \;\middle|\; \epsilon \in E \right\}. \quad (1)$$

Equation (1) defines a discrete-time, control-affine system, which we take directly as the object of verification. Note that when computing the next value of $\vec{x}[i]$, f_i and $\vec{u}$ may depend on the entire previous state $\vec{x}$, not only on the component $\vec{x}[i]$. We assume that the neural network controller $\vec{u}$ is a fully-connected, feed-forward network with n inputs, n outputs, and ReLU activations.

The *preimage* (i.e., the set of predecessors) of a set of states $\mathcal{X}$, and must be treated with care due to the presence of nondeterminism. Adversarial perturbations induce two distinct notions of predecessors: those that *must* reach $\mathcal{X}$, and those that *may* reach $\mathcal{X}$. Let $next^{\mathcal{N}}(\cdot, t)$ denote the set of states reachable by t successive applications of the next-state relation. The set of states that *must* reach $\mathcal{X}$ in t steps is defined as

$$prev_G^{\mathcal{N}}(\mathcal{X}, t) \;=\; \left\{ \vec{y} \in \mathbb{R}^n \;\middle|\; next^{\mathcal{N}}(\vec{y}, t) \subseteq \mathcal{X} \right\}. \quad (2)$$

That is, every state reachable from $\vec{y}$ in exactly t steps lies in $\mathcal{X}$. Conversely, the set of states that *may* reach $\mathcal{X}$ in t steps is defined as

$$prev_A^{\mathcal{N}}(\mathcal{X}, t) \;=\; \left\{ \vec{y} \in \mathbb{R}^n \;\middle|\; next^{\mathcal{N}}(\vec{y}, t) \cap \mathcal{X} \neq \emptyset \right\}. \quad (3)$$

That is, at least one state reachable from $\vec{y}$ in t steps lies in $\mathcal{X}$. The choice between these previous-state operators depends on the property being verified, and we elaborate on this distinction in the following section.

B. Reach-Avoid Properties

Let $\mathcal{X}_0$ be a set of initial states. We define a forward trajectory $\mathbf{F}^{\mathcal{N}}(\mathcal{X}_0)$, as the sequence of state sets $(\mathcal{X}_0, \mathcal{X}_1, \dots, \mathcal{X}_T)$, where $\mathcal{X}_i = next^{\mathcal{N}}(\mathcal{X}_{i-1})$ for all $i \in [T]$. Similarly, if $\mathcal{X}_T$ is a target, we define the *guaranteed* backward trajectory $\mathbf{B}_G^{\mathcal{N}}(\mathcal{X}_T)$ as the sequence of state sets $(\mathcal{X}_T, \mathcal{X}_{T-1}, \dots, \mathcal{X}_0)$, where $\mathcal{X}_i = prev_G^{\mathcal{N}}(\mathcal{X}_T, T - i)$ for all $i \in [0..T-1]$. The *allowable* backward trajectory $\mathbf{B}_A^{\mathcal{N}}(\mathcal{X}_T)$ is defined analagously, with $prev_G^{\mathcal{N}}$ replaced by $prev_A^{\mathcal{N}}$.

We say that a system $\mathcal{N}$ is *safe* if it satisfies either the forward or backward reach-avoid properties. The *forward reach-avoid properties* are given by (4):

$$\forall\, \vec{x} \in I.\, \exists\, t \in [0..T].\, \mathbf{F}^{\mathcal{N}}(\{\vec{x}\})[t] \subseteq G, \quad (4a)$$

$$\forall\, t \in [0..T].\, \mathbf{F}^{\mathcal{N}}(I)[t] \cap A = \emptyset, \quad (4b)$$

and the *backward reach-avoid properties* are given by (5):

$$\exists\, t \in [0..T].\, \mathbf{B}_G^{\mathcal{N}}(G)[t] \supseteq I, \quad (5a)$$

$$\forall\, t \in [0..T].\, \mathbf{B}_A^{\mathcal{N}}(A)[t] \cap I = \emptyset. \quad (5b)$$

Properties (4a) and (5a) are *reach* properties. They ensure that every forward trajectory starting from a state in the initial set eventually reaches the goal set. Conversely, Properties (4b) and (5b) are the *avoid* properties, which ensure that all possible system trajectories avoid unsafe states at every time step within the given horizon.

Properties (4) and (5) are each individually *sufficient* to establish the safety of a system, but relying solely on either forward or backward analysis is limited by the challenges discussed earlier. Our verification strategy jointly leverages (4) and (5) to soundly verify the safety of arbitrary nonlinear neural feedback systems. In particular, FABRIC establishes the stronger property (5a), which implies (4a).

C. Polyhedral Enclosures

We use the polyhedral enclosure abstraction introduced in [3] and briefly summarize the aspects needed for our development; we refer readers to the extended version of that work[1] for details, including full definitions and proofs. A *bounding set* $B = \langle n, P, L, U \rangle$ consists of an affinely independent point set $P \subseteq \mathbb{R}^n$ together with lower and upper bounding functions L, U defined on P. From such a bounding set and a Delaunay triangulation Δ of P, we obtain a collection of $(n+1)$-dimensional polyhedra by lifting each vertex $\vec{p} \in P$ to $(\vec{p}, L(\vec{p}))$ and $(\vec{p}, U(\vec{p}))$ and taking convex hulls over the vertices of each n-simplex in Δ. The resulting union of polyhedra forms a *polyhedral enclosure* $\mathcal{E}(B, \Delta) \subseteq \mathbb{R}^{n+1}$. A bounding set is said to *enclose* a function $f : \mathbb{R}^n \to \mathbb{R}$ if f lies within this enclosure. As shown in [3], such an enclosure can be constructed for a broad class of nonlinear functions and can be encoded as a mixed-integer linear program (MILP) whose feasible region corresponds exactly to the enclosure. Enclosures can thus be used as tight but conservative abstractions to ensure that a NFS with nonlinear dynamics satisfies specific constraints.

[1] https://arxiv.org/pdf/2601.08065

III. Backward Reachability Analysis

In prior work [7], *backward reachable sets* are defined as the sets of states that reach a target under *any* admissible control, while *backward projection sets* are those that reach the target under the specific control policy implemented by the neural network. This distinction is essential in [7] because the proposed methods require *a priori* bounds on the controller outputs in order to relax the network over a bounded domain. Such bounds are an artifact of the analysis rather than a constraint of the problem, and our algorithm requires none. Accordingly, we use *backward reachable set* to denote the set of states that reach a target set under the specific neural network controller policy. We further refine this terminology based on two (*must* vs *may*) notions of backward reachable sets. Specifically, we call a set satisfying Equation (3) a *backward may-reachable set* or *may-BRS* and one satisfying Equation (2) a *backward must-reachable set* or *must-BRS*.

We consider nonlinear neural feedback systems, and therefore rely on approximations of the sets defined by Equations (1) to (3). For forward reachability analysis, an overapproximation of Equation (1) does not compromise the soundness of Properties (4a) and (4b). For backward analysis, however, an overapproximation of Equation (3) can only be used with Property (5b). To maintain the soundness of Property (5a), an underapproximation of Equation (2) is required. For ease of exposition, we refer to overapproximations of backward may-reachable sets as *outer sets* or *outer approximations* and underapproximations of backward must-reachable sets as *inner sets* or *inner approximations*. In the following subsections, we describe methods for computing both outer and inner approximations.

A. Outer Sets

Our objective is to develop algorithms that soundly compute overapproximations of the set defined by (3) in a tractable manner. Inspired by the procedure described in [7], we formulate the following problem. Given a NFS $\mathcal{N}$ with operating domain D and target set $\mathcal{X}$, we compute bounds on the solution of (3) by solving a series of mixed-integer linear programs. We first employ the technique of [3] to obtain sound polyhedral enclosures for the nonlinear transition dynamics. Then, for each $i \in [n]$, we solve the following MILP in order to get a lower bound on $\vec{x}[i]$.

$$\min \ \vec{x}[i], \tag{6a}$$

$$\text{subject to} \quad \vec{x}' = \vec{x} + \left(\vec{y} + \vec{u} + \vec{\epsilon}\right) \cdot \delta, \tag{6b}$$

$$\mathcal{M}_0, \ldots, \mathcal{M}_n, \quad \vec{\epsilon} \in E, \quad \vec{x}' \in \mathcal{X}, \quad \vec{x} \in D \tag{6c}$$

where $\mathcal{M}_1, \ldots, \mathcal{M}_n$ are MILPs whose feasible sets are the polyhedral enclosures $\mathcal{E}(B_i, \Delta_i) \supseteq \{(\vec{x}, f_i(\vec{x})) \mid \vec{x} \in D\}$, with input and output variables $(\vec{x}, \vec{y}[1]), \ldots, (\vec{x}, \vec{y}[n])$, and $\mathcal{M}_0$ is an exact MILP formulation of the fully-connected ReLU-activated neural network controller using standard methods [11], with input and output variables $(\vec{x}, \vec{u})$. Replacing the minimization with a maximization yields corresponding upper bounds. Together, these bounds define a hyperrectangle that is an outer set, a sound overapproximation of the true may-BRS of $\mathcal{X}$. This outer set is sound but conservative. The conservatism arises from the fact that we typically have extremely loose bounds on $\vec{x}$: for soundness, we use the domain D to bound $\vec{x}$, but D may substantially overapproximate the set of states that are actually backward reachable.

The method of [7] mitigates this issue by assuming *a priori* bounds on the controller outputs and restricting the domain of the backward reachability problem accordingly. However, obtaining tight controller bounds is often difficult in practice, and loose bounds can lead to highly conservative overapproximations. Subsequent work [10] removes this requirement by introducing the *DRIP* domain refinement algorithm, which iteratively tightens the domain using previously computed overapproximations of the backward reachable set. While DRIP uses the CROWN framework [12] and is formulated for systems with linear transition functions, it cannot be directly applied to nonlinear dynamics. In this work, we extend DRIP to neural feedback systems with nonlinear transition functions (see Algorithm 1).

Algorithm 1 first constructs MILPs encoding an overapproximation of $\vec{u}$ and polyhedral enclosures of each $f_i \in \vec{F}$ using the techniques of [3]. It then solves n instances of (6) to obtain lower bounds $\vec{L}$ and n corresponding maximization problems to obtain upper bounds $\vec{U}$, defining a refined hyperrectangular domain D. This process repeats until the volume improvement falls below a threshold α or a maximum number of iterations $K_{\max}$ is reached. The procedure extends naturally to k time steps by iteratively replacing the target set $\mathcal{X}$ with the previously computed overapproximate may-BRS D. Following [13], we refer to this as *concrete* overapproximate backward reachability analysis. The refinement is monotone. Since (6) constrains $\vec{x} \in D$, the hyperrectangle formed by the resulting bounds is contained in D, so the domains form a nested sequence bounded below by the true may-BRS. Each iterate is a sound outer set, and so α and $K_{\max}$ bound the effort spent on refinement.

The concrete approach can induce excessive conservatism, mirroring behavior observed in concrete forward reachability analysis [3]. To mitigate this effect, we extend the symbolic multi-step approach of [3], [13] by solving for multiple time steps simultaneously. We define the k-step problem, for arbitrary $k \in \mathbb{N}$, as follows. Let $\vec{x}^{(0)}, \ldots, \vec{x}^{(k)}$ denote the states at the $k+1$ time points under consideration, so that the target is reached at $\vec{x}^{(k)}$ and we seek bounds on $\vec{x}^{(0)}$; a superscript (j) denotes the copy of a variable or constraint system instantiated at time step j. We solve, for each $i \in [n]$:

$$\begin{aligned} &\min \ \vec{x}^{(0)}[i], \\ &\text{subject to} \ \ \vec{x}^{(j)} = \vec{x}^{(j-1)} + \left(\vec{y}^{(j-1)} + \vec{u}^{(j-1)} + \vec{\epsilon}^{(j-1)}\right) \cdot \delta, \\ &\mathcal{M}_0^{(j-1)}, \ldots, \mathcal{M}_n^{(j-1)}, \ \vec{\epsilon}^{(j-1)} \in E, \ \vec{x}^{(j-1)} \in D, \ \vec{x}^{(k)} \in \mathcal{X} \end{aligned} \tag{7}$$

where the constraints are imposed for each $j \in [k]$. As before, we can compute an upper bound by replacing the minimization with a maximization. We can also run Algo-

Algorithm 1 Domain Refinement with Polyhedral Enclosures (DRiPy)

Require: Target set $\mathcal{X}$, neural feedback system $\mathcal{N}$, maximum number of iterations $K_{\max}$, improvement threshold α.
Ensure: overapproximation of the one-step may-BRS $prev_A^{\mathcal{N}}(\mathcal{X}, 1)$

$k \leftarrow 0$ ▷ Initialize iteration counter
repeat
  $\mathcal{M}_0 \leftarrow \textsc{ApproxNetwork}(\vec{u}, D)$ ▷ Construct MILP approximating controller $\vec{u}$
  $\mathcal{M}_1, \ldots, \mathcal{M}_n \leftarrow \textsc{OvertPoly}(\vec{F}, D)$ ▷ Compute MILPs for nonlinear dynamics
  $(\vec{L}, \vec{U}) \leftarrow \textsc{Solve}(\mathcal{N}, \mathcal{M}_0, \ldots, \mathcal{M}_n)$ ▷ Solve (6) to get bounds on may-BRS
  $D_{old} \leftarrow D$ ▷ Save domain D
  $D \leftarrow \textsc{Hyperrectangle}(\vec{L}, \vec{U})$ ▷ Refine D to the hyperrectangle formed by $\vec{L}$ and $\vec{U}$
  $k \leftarrow k + 1$ ▷ Increment iteration counter
until $vol(D_{old})/vol(D) - 1 < \alpha$ **or** $k \geq K_{\max}$
return D

rithm 1 with the new formulation in (7) to refine the k-step backward may-reachable set. Finally, we can implement a *hybrid* strategy by first running the concrete approach, then replacing the domain D with the computed concrete BRS at each stage to improve precision.

B. Inner Sets

By definition, any sound outer set must contain the true backward reachable set. We exploit this property to compute an underapproximation of the true must-BRS by formulating a constrained optimization problem whose objective is to identify the largest subset of the outer set whose overapproximated forward image is fully contained in the target set. This leads to the following optimization problem:

$$\max \ \textsc{vol}(\mathcal{X}^i), \tag{8a}$$
$$\text{subject to} \quad \textsc{Reach}(\mathcal{X}^i) \subseteq \mathcal{X}, \tag{8b}$$
$$\mathcal{X}^i \subseteq \mathcal{X}^o, \tag{8c}$$

where VOL denotes the volume operator, REACH denotes a sound algorithm for overapproximate forward reachability analysis, $\mathcal{X}^o$ is an outer set, and $\mathcal{X}$ is the target set. We next propose several approaches for solving this efficiently. All proposed approaches assume that the user can compute an outer set, as well as an efficient overapproximation of the forward reachable set. Because VOL and REACH are in general nonlinear, we do not solve (8) exactly: each algorithm returns a candidate set that is accepted only if a sound forward query certifies $\textsc{Reach}(\mathcal{X}^i) \subseteq \mathcal{X}$, so soundness does not depend on attaining a global optimum, and the returned sets are certified deterministically rather than probabilistically.

*1) Scaled Hyperrectangular Approximation of Reachable Polytopes (*SHARP*):* The SHARP approach computes an inner set by scaling an outer set inward. We assume that the outer set is a hyperrectangle with center $\vec{c}$ and radii $\vec{r}$, that is, $\mathcal{X}^o = \{\vec{x} \in \mathbb{R}^n \mid |\vec{x} - \vec{c}| \preccurlyeq \vec{r}\}$, where $|\cdot|$ is taken element-wise; we abbreviate this as $\mathcal{X}^o = \vec{c} \pm \vec{r}$. The inner set is defined as $\mathcal{X}^i = \vec{c} \pm \vec{\rho} \cdot \vec{r}$, where $\vec{\rho} \in (0, 1)^n$. Our objective is to estimate the largest admissible scaling vector $\vec{\rho}$ such that $\textsc{Reach}(\mathcal{X}^i)$ is fully contained in the target set. This leads to the following optimization problem:

$$\max \ \sum_{j=1}^{n} \log(\vec{\rho}[j]) \tag{9a}$$
$$\text{s.t.} \quad \mathbf{0} \preccurlyeq \vec{\rho} \preccurlyeq \mathbf{1}, \ \ \mathcal{X}^i = \vec{c} \pm \vec{\rho} \cdot \vec{r}, \ \ \textsc{Reach}(\mathcal{X}^i) \subseteq \mathcal{X}. \tag{9b}$$

We estimate the optimal $\vec{\rho}$ via an iterative line-search procedure inspired by golden-section search. We use golden-section search because it typically requires fewer containment queries than a binary search for comparable bracketing accuracy. Solving equation (9) returns a *certified* inner approximation of the true must-BRS.

This formulation has several limitations, the most significant of which are the need for frequent containment queries and the assumption that the inner set is an axis-aligned hyperrectangle centered at the center of the outer set. Nonetheless, the approach often converges quickly in practice. The formulation also typically requires a large number of forward reachability queries, which can be problematic when such queries are computationally expensive. To mitigate these limitations, we propose the following alternatives.

*2) Convex Rectangle Inferred via Sampled Positives (*CRISP*):* The CRISP approach seeks to reduce the number of forward reachability queries required to compute an inner set. To this end, we replace the scaling-based procedure with a sampling-based alternative. We draw dense, space-filling samples from the outer set. Each sample is then classified as positive or negative via rejection sampling, based on containment of its forward reachable set in the target set. Let P be the set of positive samples. Our objective is to identify the largest axis-aligned hyperrectangle contained within the convex hull of the positive samples. The resulting optimization problem is:

$$\max \quad \textsc{vol}(\mathcal{X}^i) \tag{10a}$$
$$\text{s.t.} \quad \mathcal{X}^i(\vec{c}, \vec{r}) \subseteq conv(P), \quad \vec{r} \succcurlyeq \vec{0}. \tag{10b}$$

where $\mathcal{X}^i(\vec{c}, \vec{r})$ is an axis-aligned hyperrectangle, and $conv$ is the convex hull operator. Note that this hyperrectangle may not necessarily share a center with the outer set. With an

appropriate surrogate for the VOL operator, this optimization problem can be formulated as a convex program.

While this formulation addresses several of the major limitations of the SHARP approach, it remains sensitive to irregular backward reachable sets. In particular, when the backward reachable set is nonconvex or contains holes (e.g., a toroidal set), the resulting candidates may fail the forward reachability certification step and therefore be nonviable. We mitigate this limitation via the following modification.

*3) Constrained Local Exclusion of Aligned Negatives (*CLEAN*):* The CLEAN approach is designed to handle problem instances with irregular backward reachable sets. Although such cases are uncommon in practice, we include this method for completeness. We employ the same space-filling rejection sampling algorithm used in CRISP, but here we retain the set of negative samples N. Our objective is to identify the largest axis-aligned hyperrectangle that contains no negative samples.

$$\max \quad \text{VOL}(\mathcal{X}^i) \tag{11a}$$

$$\text{s.t.} \;\; \mathcal{X}^i(\vec{c},\vec{r}) \cap N = \emptyset \;\; \mathcal{X}^i(\vec{c},\vec{r}) \subseteq \mathcal{X}^o, \;\; \vec{r} \succcurlyeq \vec{0}. \tag{11b}$$

The computational complexity of this problem depends on both the surrogate used for the volume objective and the representation of the emptiness constraint in eq. (11b). While a convex surrogate may be used for the volume objective, enforcing the emptiness constraint requires integer variables. In particular, binary variables are needed to enforce exclusion along at least one dimension for each negative sample $n \in N$. Consequently, the resulting optimization problem is, in the general case, a mixed-integer linear program. The number of binary variables scales with both the system dimension and the number of negative samples.

Algorithm 2 Fast Inner Template Sets (FITS)

Require: Target set $\mathcal{X}$, neural feedback system $\mathcal{N}$, outer set $\mathcal{X}^o$, maximum number of iterations $K_{\max}$, inner approach $p \in \{\text{CLEAN}, \text{CRISP}\}$

Ensure: underapproximation of must-BRS $prev_G(\mathcal{X}, 1)$

$k \leftarrow 0$ ▷ Initialize iteration counter
repeat
  INNER $\leftarrow$ SELECT(p) ▷ Select inner approximation
  $\mathcal{X}^i \leftarrow$ INNER$(\mathcal{N}, \mathcal{X}^o)$ ▷ Approximate the solution
  if REACH$(\mathcal{X}^i) \subseteq \mathcal{X}$ **then**
    return $\mathcal{X}^i$ ▷ Accept candidate
  end if
  $k \leftarrow k + 1$ ▷ Increment iteration counter
  $\mathcal{X}^o \leftarrow \mathcal{X}^i$ ▷ Update outer set
until $k \geq K_{\max}$
return $\emptyset$

Solving the optimization problem in equation (10) or (11) yields a *candidate* underapproximate solution to the must-BRS. These optimization problems are used as subroutines in Algorithm 2 for computing an inner set. We can further optimize Algorithm 2 by sampling a finite set of points from the candidate set $\mathcal{X}^i$ and iteratively reducing the set until no negative samples remain *before* performing the forward reachability query.

As in the overapproximation case, the algorithm can be extended to k time steps by calling the algorithm k times, each time replacing the target set $\mathcal{X}$ by the underapproximated must-BRS $\mathcal{X}^i$ calculated in the previous call. We refer to the resulting procedure as a *concrete* underapproximation if candidate sets are validated using a concrete forward reachability query, and as *symbolic* underapproximation if validation is instead performed using a symbolic query.

IV. The FaBRIC Verification Strategy

Rather than exclusively computing forward or backward reachable sets, we propose verifying $\mathcal{N}$ by combining both analyses. Specifically, we partition the planning horizon into $T = F + B$ steps, where F denotes the number of forward-analysis steps and B the number of backward-analysis steps. This yields the **F**orward **a**nd **B**ackward **R**eachability **I**ntegration for **C**ertification FaBRIC strategy. The subdivision is treated as a configurable parameter of the strategy. We generally expect $F > B$ because the forward problem is implicitly encoded within the backward problem; consequently, the computational effort required to perform $k \in \mathbb{N}$ steps of backward analysis typically exceeds that required for k steps of forward analysis. This discrepancy is even more pronounced when verifying reach properties, since the repeated certification queries introduce additional overhead. A small B nonetheless improves on forward-only analysis because forward overapproximation error compounds with the horizon. Replacing the final, least precise forward steps with backward analysis from a compact target removes the most expensive part of the forward computation. In this work, we consider subdivisions satisfying $F \geq 0.75\,T$ for all problems and leave the meta-optimization of F and B to future work.

To establish that the system satisfies a reach property (Properties 4a and 5a), we perform F steps of forward analysis from I and B steps of backward analysis from G. If the overapproximation of the forward reachable set at time step F is contained within the underapproximation of the backward must-reachable set, then the system is guaranteed to reach the goal set.

To verify that the system satisfies an avoid property (Properties 4b and 5b), we compute F steps of overapproximated forward analysis from the initial set and B steps of overapproximated backward analysis from the avoid set A. Let $U = \cup_{t=0}^{B} prev_A(A)$. If each of the forward sets at time $t \in [0, F]$ is disjoint from A and the forward set at time F is additionally disjoint from U, then the system is guaranteed to avoid all unsafe states, thereby ensuring safety.

V. Evaluation

We evaluate our algorithms and overall strategy on three classes of neural feedback systems drawn from the ARCH Competition benchmark suite [14]. For each benchmark, we consider controllers inspired by those used in the original ARCH Competition, which are obtained by training neural

TABLE I

COMPARING DRIPY TO HYBREACH-MILP. TIMES IN SECONDS; VOL. IS THE OUTER-SET VOLUME AFTER FIVE BACKWARD STEPS. IMPROVEMENT IS THE SIGNED RATIO TO UNREFINED DRIPY: $r > 0$ IS $r\times$ BETTER, $r < 0$ IS $|r|\times$ WORSE.

	HyBReach-MILP		DRIPY (★)		DRIPY (★‡)		Improvement (↑)	
	Time (s)	Vol.	Time (s)	Vol.	Time (s)	Vol.	Time	Vol.
Tora (sma.)	**1.14**	1.44E-01	1.63	1.67E-01	5.17	**4.16**E-02	−1.43	−1.16
Tora (med.)	**1.14**	1.39E-01	2.63	1.57E-01	5.20	**4.11**E-02	−2.22	−1.13
Tora (larg.)	**1.40**	1.44E-01	2.53	1.66E-01	6.68	**4.16**E-02	−1.81	−1.15
Unicycle (sma.)	5501.0	1.84E+03	**60.59**	1.51E+02	1897.94	**4.00**E+01	90.79	12.23
Unicycle (med.)	4980.90	1.70E+03	**55.04**	1.51E+02	124.913	**4.20**E+01	90.50	11.30
Unicycle (larg.)	4739.67	1.82E+03	**100.58**	1.51E+02	1611.40	**4.20**E+01	47.12	12.06
Attitude (sma.)	1871.68	3.73E+05	**77.75**	2.57E+04	1165.71	**1.25**E+02	24.07	14.51
Attitude (med.)	1902.81	3.73E+05	**91.46**	2.57E+04	1060.17	**1.25**E+02	20.81	14.51
Attitude (larg.)	2256.76	3.73E+05	**145.53**	2.57E+04	626.70	**1.25**E+02	15.51	14.52

policies via imitation learning from a model predictive control (MPC) expert.

For each system, we train policies with $0.5\times$, $1\times$, and $2\times$ the number of neurons used in the corresponding ARCH benchmark controller, while retaining all other architectural features. We refer to these variants as the small, medium, and large policies respectively. We provide summaries below, and detailed descriptions in the extended version.

A. Benchmarks

TORA requires verifying that the states of a four-dimensional actuated cart model remain within a safe region over a specified time interval. *Unicycle* and *Attitude Control* require verifying the safety of a four-dimensional autonomous vehicle model and a six-dimensional rigid-body aircraft model.

B. Baselines

We compare our method for computing outer approximations against HyBreach-MILP, a state-of-the-art approach for computing outer reachable sets of nonlinear neural feedback systems [7]. For HyBreach-MILP, we compute bounds on the admissible control inputs via parameter search and report results obtained using the two-step procedure described in [7]. We compare our method for computing inner approximations against BURNS, the only existing tool for computing inner reachable sets for nonlinear neural feedback systems [15].[2] We report the resulting comparisons below. Finally, we compare the FABRIC strategy against the forward reachability analysis method introduced in [3][3] . This algorithm represents the state of the art in combinatorial forward reachability for nonlinear neural feedback systems and therefore provides a strong baseline for evaluating our approach.

C. Implementation Details

The convexity of equations (10) and (11) depends on the surrogates used to compute the VOL operator. For simplicity, we assume that the reachable sets are axis-aligned hyper-rectangles and define the volume surrogate as the sum of the radii $\vec{r}$. We generate samples using deterministic Sobol sequences [16]. This provides an efficient space-filling sampling scheme suitable for our rejection-sampling procedure. We use the Gurobi solver [17] for all optimization queries.

D. Experimental Setup

All computation times were obtained by running each tool on an AMD Ryzen 9 7950X processor. For soundness, all reported results correspond to certified bounds on the optimal value of the objective function. Since all methods rely on the OVERT algorithm, we use a fixed parameter configuration with $N = 2$, a relative error tolerance of 5×10^{-3}, and an absolute error tolerance of 10^{-4} across all experiments. For the Attitude benchmark, we instead use $N = 1$ for both HyBreach-MILP and BURNS, as these tools otherwise time out.

For all backward reachability benchmarks, we perform five steps of symbolic backward reachability analysis using the corresponding avoid and goal sets. For the overapproximation benchmarks, we impose a per–solver-call timeout of 900 seconds for each tool, set $K_{\max} = 10$, and use $\alpha = 0.01$. For the underapproximation benchmarks, we set $K_{\max} = 10$ and report the best result across 10 independent attempts for both SHARP and BURNS. We impose a per–solver-call timeout of 600 seconds for each algorithm, together with an additional timeout of 300 seconds for forward reachability queries.

For the FABRIC benchmarks, we employ a combination of concrete and symbolic reachability for the forward-analysis component, together with purely symbolic reachability for the backward component. For the TORA benchmark with $T^{\mathcal{T}} = 20$, we set $F = 15$ and $B = 5$, and further subdivide the forward component into two hybrid symbolic runs with horizons $T = 10$ and $T = 5$, respectively. For the Unicycle benchmark with $T^{\mathcal{U}} = 50$, we set $F = 47$ and $B = 3$, further subdividing the forward component into four hybrid symbolic runs with horizon $T = 10$ and one run with horizon $T = 7$. For the Attitude benchmark with $T^{\mathcal{R}} = 10$, we set $F = 8$ and $B = 2$. We impose a

[2]Our comparison is based on our own re-implementation of BURNS based on the algorithm described in [15], as the actual code used in that work was not available at the time we were preparing this work.

[3]We use an existing public implementation provided by the authors of [3].

TABLE II

COMPARING OUR SHARP AND FITS ALGORITHMS TO BURNS. TIMES IN SECONDS; VOL. IS THE INNER-SET VOLUME AFTER FIVE BACKWARD STEPS. −: ZERO-VOLUME SET RETURNED; ×: NO RESULT RETURNED.

	BURNS		SHARP (★)		CRISP (★)		CLEAN (★)	
	Time (s)	Vol.	Time (s)	Vol.	Time (s)	Vol.	Time (s)	Vol.
Tora (sma.)	32.59	**6.35**E-04	**6.62**	1.85E-05	8.52	3.82E-04	6.82	2.41E-06
Tora (med.)	32.59	**6.35**E-04	**6.72**	1.85E-05	8.83	3.82E-04	6.79	2.40E-06
Tora (lar.)	65.92	**4.70**E-04	10.83	7.66E-05	14.15	3.84E-04	**10.32**	2.35E-06
Unicycle (sma.)	606.11	–	157.92	5.88E-03	136.52	**1.94**E-02	**50.18**	1.67E-02
Unicycle (med.)	1524.04	–	126.33	5.89E-03	116.70	**3.43**E-02	**36.41**	1.47E-02
Unicycle (lar.)	1001.18	–	219.07	5.85E-03	190.62	8.59E-03	**73.17**	**1.61**E-02
Attitude (sma.)	557.85	–	18040.96	1.31E-04	**5297.40**	**6.86**E-04	13828.22	4.05E-04
Attitude (med.)	×	×	18528.36	1.31E-04	**5724.30**	**6.68**E-04	14279.95	4.47E-04
Attitude (lar.)	×	×	17834.08	1.31E-04	**5400.88**	**6.83**E-04	13454.96	4.05E-04

per-solver call timeout of 600 seconds for all forward and backward analyses. We fix these hyperparameters across all benchmarks rather than tuning them per instance and do not report a sensitivity analysis; our results therefore reflect a single untuned configuration rather than the best achievable for any individual tool.

TABLE III

COMPARING FABRIC TO FORWARD-ONLY ANALYSIS. TIMES IN SECONDS. IMPROVEMENT IS THE SIGNED SPEEDUP: $r > 0$ MEANS FABRIC IS $r\times$ FASTER, $r < 0$ THAT IT IS $|r|\times$ SLOWER.

	Forward	FABRIC(★)		Improvement (↑)	
	Analysis	Reach	Avoid	Reach	Avoid
Tora (Small)	1.99	10.87	6.41	−5.46	−3.22
Tora (Med)	2.21	10.09	6.61	−4.57	−2.99
Tora (Large)	3.29	16.70	8.92	−5.08	−2.71
Unicycle (Small)	1302.98	280.51	292.02	4.65	4.46
Unicycle (Med)	1157.98	261.27	260.15	4.43	4.45
Unicycle (Large)	2928.21	384.56	408.82	7.61	7.16
Attitude (Small)	6980.36	8365.28	4001.11	−1.20	1.75
Attitude (Med)	5823.11	7349.67	3458.57	−1.26	1.68
Attitude (Large)	7223.28	10658.96	4182.43	−1.48	1.73

VI. RESULTS

A. Computing Outer Sets

We report results comparing HyBReach-MILP with the DRIPY algorithm in Table I. The black star (★) denotes our polyhedral enclosure based algorithms, and the dagger (‡) denotes the modified refinement scheme. To enable a fair comparison, we present results for DRIPY both with and without domain refinement. We additionally report the improvement in computation time and set volume achieved by DRIPY relative to HyBReach-MILP when no refinement steps are applied.

The TORA benchmark is the easiest among the considered benchmarks. All tools compute outer approximations in under 10 seconds, with HyBReach-MILP outperforming DRIPY in terms of computation time. HyBReach-MILP also produces a tighter outer approximation than unrefined DRIPY. Despite this advantage, DRIPY with domain refinement computes an outer set that is $3.5\times$ smaller in under 10 seconds.

The Unicycle and Attitude benchmarks exhibit a different performance trend. In these cases, the computational cost of HyBReach-MILP outweighs its precision advantage, and unrefined DRIPY becomes the fastest method. In some instances, unrefined DRIPY is nearly two orders of magnitude faster than HyBReach-MILP, while still achieving an order-of-magnitude reduction in outer set volume. These results are partly explained by the timeout imposed on all tools: HyBReach-MILP frequently times out, resulting in larger cumulative computation times and looser outer sets, since we report objective bounds. The medium Unicycle benchmark further highlights the benefits of domain refinement. In this setting, DRIPY requires substantially fewer refinement iterations to converge, resulting in significantly lower computation times compared to other variants.

For the Unicycle benchmark, domain refinement reduces the outer-set volume by a factor of 3.5 relative to unrefined DRIPY. Compared to HyBReach-MILP, domain refinement produces an outer set that is over $40\times$ smaller while requiring less computation time. For the Attitude benchmark, domain refinement yields an additional reduction factor of approximately 200 relative to unrefined DRIPY. Compared to HyBReach-MILP, domain refinement produces an outer set that is over $2900\times$ smaller while also requiring less computation time.

B. Computing Inner Sets

By design, the sets returned by our methods (SHARP, CRISP, CLEAN) are certified via a runtime check, whereas those returned by BURNS are certified by construction. Our approach trades a modest reduction in set volume for the assurance of runtime certification.

On the TORA benchmark, BURNS outperforms all variants of our algorithm in terms of inner set volume, although we are able to return certified inner sets in a fraction of the computation time. The sets returned by BURNS are nearly $2\times$ larger than the largest set returned by SHARP or FITS.

This trend does not persist on the more challenging benchmarks. On these benchmarks, BURNS is unable to compute a valid inner set within $K_{\max}$ attempts, whereas

all variants of the FITS algorithm succeed. For the Unicycle benchmark, BURNS completes execution but is unable to return an inner set with nonzero volume. In contrast, for the Attitude benchmark, BURNS struggles to identify feasible points in the inner set. This difficulty arises largely from the need to sample a sparse set of points in a six-dimensional domain. Our reported results use up to 5×10^7 samples; we were unable to use larger sample counts due to memory limitations. Our sampling-based approaches avoid this limitation by sampling from the outer set, which constitutes a substantially smaller subset of the domain. Notably, the CRISP algorithm is the best-performing FITS variant across all benchmarks, while the CLEAN algorithm produces the largest inner set on a single instance. The SHARP algorithm struggles across all benchmarks but consistently returns a valid, albeit small, inner set.

C. Combining Forward and Backward Analysis

We combine the forward analysis method of [3] with refined DRIPY and the CRISP variant of FITS, and compare against forward analysis alone.

A single run of forward analysis suffices to verify the forward reach–avoid properties (eqs. (4a) and (4b)). In contrast, verifying the backward reach–avoid properties (eqs. (5a) and (5b)) requires two separate backward-analysis runs, corresponding to the computation of the must-BRS and the may-BRS. We report these results separately.

On relatively easy benchmarks such as TORA, the overhead introduced by backward analysis dominates the overall computational cost, rendering the FABRIC strategy detrimental. As the benchmarks become more challenging, however, the FABRIC strategy becomes beneficial. On the Unicycle benchmark, we observe speedups of up to $7\times$ for both reach and avoid properties. Even after accounting for the fact that forward-only verification requires only a single forward-analysis run, the FABRIC strategy still improves over forward-only analysis by a factor of 2–$3\times$. The results are mixed on the Attitude benchmark, where we observe improvements of up to $1.7\times$ on the avoid specification, but the FABRIC strategy appears detrimental for the reach specification. When we account for the need for a single forward-only analysis run, FABRIC requires up to $2\times$ the amount of time needed to verify the same set of properties.

VII. Conclusions

We introduced FABRIC, a new strategy for verifying reach–avoid specifications in nonlinear neural feedback systems. The FABRIC strategy is enabled by a set of new algorithms that soundly approximate may- and must-backward reachable sets for nonlinear neural feedback systems. Using a representative benchmark suite, we demonstrated that our backward reachable set approximation algorithms significantly improve on the prior state of the art. Finally, we showed that the FABRIC strategy can substantially reduce the computational effort required to verify reach–avoid specifications in neural feedback systems.

a) Future Work: We plan to explore further optimizations for inner-set estimation, as the current methods remain sensitive to the curse of dimensionality. One promising direction is to use sampling to refine the domain prior to running the FITS algorithm. We also intend to investigate the meta-optimization problem of selecting F and B, with the goal of automating this selection process, and to conduct a systematic sensitivity analysis of the remaining hyperparameters.

References

[1] N. Csomay-Shanklin, W. D. Compton, I. D. J. Rodriguez, E. R. Ambrose, Y. Yue, and A. D. Ames, "Robust agility via learned zero dynamics policies," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 9116–9123.

[2] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.

[3] S. I. Akinwande, C. Sidrane, M. J. Kochenderfer, and C. Barrett, "Verifying nonlinear neural feedback systems using polyhedral enclosures," *arXiv preprint arXiv:2503.22660*, 2025.

[4] H. Delecki, M. R. Schlichting, M. Arief, A. Corso, M. Vazquez-Chanlatte, and M. J. Kochenderfer, "Diffusion-based failure sampling for evaluating safety-critical autonomous systems," in *2025 IEEE Engineering Reliable Autonomous Systems (ERAS)*. IEEE, 2025, pp. 1–8.

[5] U. Mandal, G. Amir, H. Wu, I. Daukantas, F. Newell, U. Ravaioli, B. Meng, M. Durling, M. Ganai, T. Shim *et al.*, "Formally verifying deep reinforcement learning controllers with lyapunov barrier certificates," in *CONFERENCE ON FORMAL METHODS IN COMPUTER-AIDED DESIGN–FMCAD 2024*, 2024, p. 95.

[6] M. Althoff, G. Frehse, and A. Girard, "Set propagation techniques for reachability analysis," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 369–395, 2021.

[7] N. Rober, S. M. Katz, C. Sidrane, E. Yel, M. Everett, M. J. Kochenderfer, and J. P. How, "Backward reachability analysis of neural feedback loops: Techniques for linear and nonlinear systems," *IEEE Open Journal of Control Systems*, vol. 2, pp. 108–124, 2023.

[8] Y. Zhang, H. Zhang, and X. Xu, "Backward reachability analysis of neural feedback systems using hybrid zonotopes," *IEEE Control Systems Letters*, vol. 7, pp. 2779–2784, 2023.

[9] ——, "Reachability analysis of neural network control systems with tunable accuracy and efficiency," *IEEE Control Systems Letters*, vol. 8, pp. 1697–1702, 2024.

[10] M. Everett, R. Bunel, and S. Omidshafiei, "Drip: Domain refinement iteration with polytopes for backward reachability analysis of neural feedback loops," *IEEE Control Systems Letters*, vol. 7, pp. 1622–1627, 2023.

[11] V. Tjeng, K. Xiao, and R. Tedrake, "Evaluating robustness of neural networks with mixed integer programming," *arXiv preprint arXiv:1711.07356*, 2017.

[12] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient neural network robustness certification with general activation functions," *Advances in neural information processing systems*, vol. 31, 2018.

[13] C. Sidrane, A. Maleki, A. Irfan, and M. J. Kochenderfer, "Overt: An algorithm for safety verification of neural network control policies for nonlinear systems," *Journal of Machine Learning Research*, vol. 23, no. 117, pp. 1–45, 2022.

[14] D. Manzanas_Lopez, M. Althoff, L. Benet, C. Blab, M. Forets, Y. Jia, T. T. Johnson, M. Kranzl, T. Ladner, L. Linauer *et al.*, "Arch-comp24 category report: Artificial intelligence and neural network control systems (ainncs) for continuous and hybrid systems plants," 2024.

[15] C. Sidrane and J. Tumova, "Burns: Backward underapproximate reachability for neural-feedback-loop systems," *arXiv preprint arXiv:2505.03643*, 2025.

[16] I. M. Sobol, "Distribution of points in a cube and approximate evaluation of integrals," *USSR Computational mathematics and mathematical physics*, vol. 7, pp. 86–112, 1967.

[17] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2026. [Online]. Available: https://www.gurobi.com